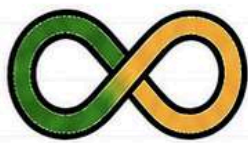




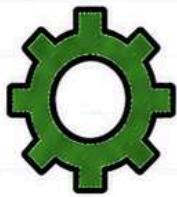
@qa_insights

DEVOPS

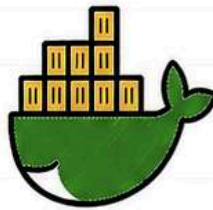
INTERVIEW PREPARATION KIT



DEVOPS
FUNDAMENTALS



CI / CD
PIPELINES



DOCKER &
CONTAINERS



KUBERNETES



CLOUD
PLATFORMS

SWIPE TO NEXT



FOLLOW



SHARE



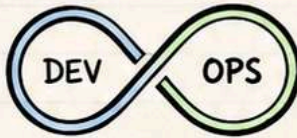
COMMENT



LIKE

BASIC TERMINOLOGY

1 What is DevOps, and why is it important?

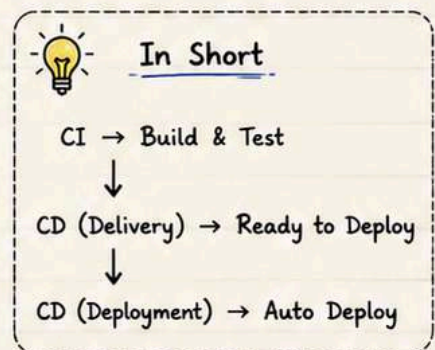


- DevOps is a culture and set of practices that bring Development (Dev) and Operations (Ops) together.
- Goal → Build, test, release and run software faster and more reliably.
- Important because it improves collaboration, speeds up delivery, reduces errors and increases customer satisfaction.

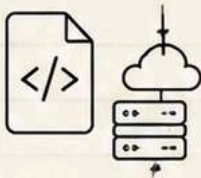
2 What is CI/CD? Explain the difference between Continuous Integration, Continuous Delivery, and Continuous Deployment.



- **CI** (Continuous Integration): Developers merge code frequently into a shared repo. Automated builds & tests validate the changes.
- **CD** (Continuous Delivery): Code is always in a deployable state. After passing tests, it's ready to be released to production.
- **CD** (Continuous Deployment): Every change that passes all tests is automatically deployed to production without manual approval.

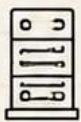


3 What is Infrastructure as Code (IaC)?



- IaC means managing and provisioning infrastructure using code instead of manual steps.
- We define servers, networks, databases, etc. in code and tools create them automatically.
- Examples: Terraform, AWS CloudFormation, Ansible.
- Benefits: Consistency, version control, automation, repeatability.

4 What is the difference between Virtual Machines and Containers?

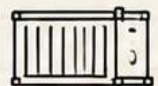


Virtual Machines (VMs)

- Each VM has its own OS, libraries and binaries.
- Heavier and slower to start.
- More resource consumption.
- Strong isolation.

Containers

- Share the host OS kernel.
- Lightweight and start in seconds.
- More efficient and portable.
- Process isolation.



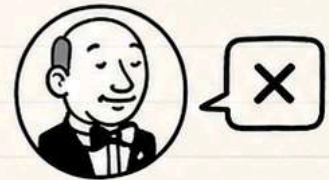
5 What is the role of a DevOps Engineer in the SDLC?



- Bridge the gap between Dev and Ops teams.
- Automate build, test, deployment and infrastructure.
- Manage CI/CD pipelines and environments.
- Monitor systems, ensure reliability, security and performance.
- Drive collaboration, continuous improvement and faster delivery.

CI/CD - SCENARIO BASED QUESTIONS

- 6 Your Jenkins pipeline suddenly starts failing after yesterday's successful deployment. How do you troubleshoot?



Troubleshooting Flow



What to do?

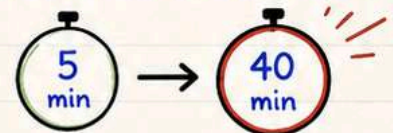
- ✓ 1) Check Build Logs: Go to the failed stage and read the console output carefully. The error message usually shows the root cause.
- ✓ 2) Identify Failed Stage: Note which stage failed (Build / Test / Deploy / Post actions) and focus there.
- ✓ 3) Recent Changes: Check code commit, Jenkinsfile changes, pipeline config updates, dependency or plugin updates.
- ✓ 4) Dependencies & Tools: Verify versions of libraries, plugins, Docker images, build tools (Maven, Node, JDK).
- ✓ 5) Environment / Agents: Confirm agent is online, has enough disk space, memory and correct permissions.
- ✓ 6) Re-run & Validate: Re-run the failed stage with more logs (--debug) or in isolation and fix the issue.



TIP:

Always compare with the last successful build. That's your best reference!

- 7 A pipeline that usually completes in 5 minutes is now taking 40 minutes. What would you investigate?



Investigation Areas

- 1. Stage Timing
- 2. Resource Usage
- 3. Recent Changes
- 4. Dependencies / Downloads
- 5. Network Issues
- 6. Caching
- 7. Parallelization
- 8. Logs & Metrics

What to check?

- ✓ 1) Stage Timing: Identify which stage/step is taking most time.
- ✓ 2) Resource Usage: Check CPU, Memory, Disk and I/O usage on build agents.
- ✓ 3) Recent Changes: New dependency, large codebase, extra tests, new scans or added steps?
- ✓ 4) Dependencies / Downloads: Large artifact downloads, npm/maven packages, Docker image pulls can slow things down.
- ✓ 5) Network Issues: Slow network, artifact repo, Docker registry or external APIs?
- ✓ 6) Caching: Is caching working? If not, everything may be re-downloaded and rebuilt.
- ✓ 7) Parallelization: Can stages or tests be run in parallel to reduce total time?
- ✓ 8) Logs & Metrics: Check timestamps, queue time vs execution time and agent performance metrics.

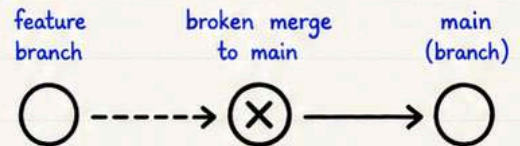


TIP:

Compare with previous successful runs and track the trend. Small changes can cause big delays!

GIT & VERSION CONTROL

- 8 A developer accidentally merged broken code into the main branch. What would you do?



Steps to Fix

- ① Assess the impact
Check what was merged and whether it's in any other branch or release.
- ↓
- ② Revert the merge
Use 'git revert' to safely undo the merge without losing history.
- ↓
- ③ Create a hotfix (if needed)
Fix the issue in a new branch from main.
- ↓
- ④ Test thoroughly
Run tests, build and verify the fix.
- ↓
- ⑤ Re-deploy
Merge the hotfix and deploy a stable build.

Key Points

- ✓ Never force push to main. It can break others' work.
- ✓ Use 'git revert -m 1 <merge_commit>' to revert a merge commit.
- ✓ Communicate with the team immediately.
- ✓ Add tests to prevent similar issues in future.
- ✓ Track the root cause and improve the process.



TIP:

Revert is safe and keeps history intact. Reset should be used only in local branches or with full team alignment.

- 9 Your release branch has merge conflicts just before production deployment. How would you handle it?



How to Handle

- ① Identify the conflicts
Run 'git status' to see conflicted files.
- ↓
- ② Understand the changes
Open each file and understand both sides of the change.
- ↓
- ③ Resolve the conflicts
Edit the files, keep correct changes and remove conflict markers.
- ↓
- ④ Test the changes
Build and run all tests to ensure nothing is broken.
- ↓
- ⑤ Commit & merge
Commit the resolved files and complete the merge.
- ↓
- ⑥ Final validation
Do a final end-to-end validation before deployment.

Best Practices

- ✓ Pull latest changes before merging.
- ✓ Resolve conflicts early, not at the last minute.
- ✓ Communicate with the team working on both branches.
- ✓ Run full regression tests after merge.
- ✓ Don't guess—understand the business impact before choosing changes.

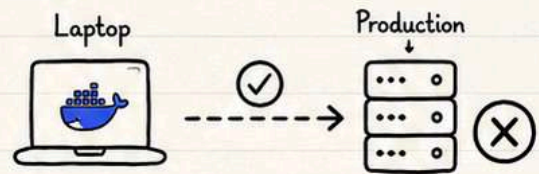


TIP:

- Small frequent merges = fewer conflicts.
- Keep release branch updated regularly.
- Use feature flags to reduce last-minute risk.

DOCKER

- 10 A Docker container works perfectly on your laptop but crashes in production. What would you check first?



What to Check First?

- ① **Check Environment Differences**
Compare OS, Docker version, kernel version, resources (CPU, Memory, Disk).
- ② **Check Logs**
Use 'docker logs <container_id>' to find the actual error.
- ③ **Check Resource Limits**
Verify memory/CPU limits, ulimits, and OOM (Out Of Memory) kills.
- ④ **Check Configuration**
Validate env variables, config files, secrets, ports, and mounts.
- ⑤ **Check Dependencies**
Ensure all external services, DBs, APIs are accessible.

Key Points

- ✓ Environment differences are the most common reason.
- ✓ Logs will tell you what actually went wrong.
- ✓ Resource limits in production are usually tighter than local.
- ✓ Config, env vars, and secrets often cause unexpected crashes.
- ✓ Always replicate the production-like environment locally if possible.



TIP:

Always compare docker inspect output between local and production containers.

- 11 Two Docker containers cannot communicate with each other. How would you troubleshoot?



Troubleshooting Checklist

- ① **Check Network**
Ensure both containers are on the same Docker network.
- ② **Check Container Status**
Verify both containers are running using 'docker ps'.
- ③ **Check Connectivity**
Use 'ping', 'curl' or 'nc' to test connectivity between containers.
- ④ **Check Ports**
Ensure the application is listening on the correct port and exposed.
- ⑤ **Check Firewall / Security**
Look for firewall rules, security groups or SELinux/AppArmor blocking traffic.
- ⑥ **Check DNS / Service Name**
Use container name or service name to connect, not localhost.
- ⑦ **Check Logs**
Review application logs for connection errors or misconfigurations.

Common Causes

- Containers are on different networks.
- Wrong IP/hostname or using localhost inside container.
- Application not listening on 0.0.0.0.
- Port not exposed or published.
- Firewall or security policies blocking traffic.
- Service dependencies not ready.



TIP:

- Use a user-defined bridge network for container-to-container communication.
- Prefer service names over IP addresses.
- Always test connectivity step by step.
- Keep logs clean and readable.

KUBERNETES

- 12** A Pod is stuck in CrashLoopBackOff.
Walk me through your troubleshooting process.



Status
CrashLoopBackOff

Troubleshooting Steps

- ① Check Pod Status
`kubectl get pod <pod> -n <ns>`
Confirm status and restarts.
- ② Describe the Pod
`kubectl describe pod <pod> -n <ns>`
Check Events for warnings or errors.
- ③ Check Logs
`kubectl logs <pod> -n <ns>`
Check application logs for the root cause.
- ④ Check Previous Logs
`kubectl logs <pod> -n <ns> --previous`
Useful if the container crashed before starting.
- ⑤ Validate Resources & Config
Check CPU/Memory limits, env vars, configmaps, secrets, volume mounts, and command/args.
- ⑥ Fix & Re-deploy
Fix the issue, update image/config and re-deploy.

Common Root Causes

- Application crash or bug
- Wrong command/entrypoint
- Missing configuration or environment variables
- Database/service dependencies unavailable
- Insufficient memory or CPU limits
- Permission issues or invalid secrets

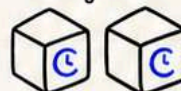


TIP:

- Always start with logs.
- Events in 'describe' output give great clues.

- 13** Some Pods are in Pending state while others are running successfully. Why?

Pending Pods



Running Pods



Why Pods Stay in Pending?

- ① Insufficient Resources
Not enough CPU, Memory or ephemeral-storage.
- ② Node Selector / Affinity mismatch
No nodes match the required labels or affinity rules.
- ③ Taints and Tolerations
Pod doesn't tolerate the taints on available nodes.
- ④ Image Pull Issues
Cannot pull image (wrong image, repo auth, or network issue).
- ⑤ PVC / Storage Issues
PVC is not bound or storage class is unavailable.
- ⑥ CNI / Network Issues
Network plugin not ready or IP address exhaustion.

How to Troubleshoot?

- ✓ `kubectl get pods -A` → see which pods are Pending.
- ✓ `kubectl describe pod <pod> -n <ns>` → check events.
- ✓ Check node status → `kubectl get nodes`.
- ✓ Verify resource quotas and limits.
- ✓ Check taints → `kubectl describe node <node>`.
- ✓ Verify PVC → `kubectl get pvc -A`.
- ✓ Check image pull secrets and registry access.
- ✓ Check CNI logs if networking is the issue.

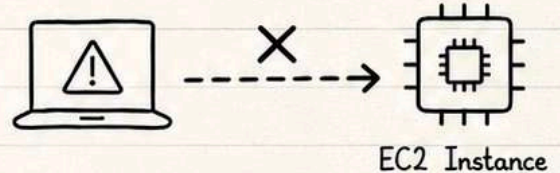


TIP:

- Events section is your best friend.
- It tells you exactly why the pod is pending.

AWS

- 14 An EC2 instance suddenly becomes unreachable. How would you identify the cause?



Troubleshooting Steps

- ① **Check Instance Status**
Go to EC2 Dashboard and verify if the instance is running.
- ② **Check Security Group**
Ensure inbound rules allow your IP on the required port (e.g., 22, 80, 443).
- ③ **Check NACL**
Verify NACL allows inbound and outbound traffic.
- ④ **Check Network / Internet**
Ensure instance has public IP, route to Internet Gateway and correct subnet.
- ⑤ **Check System / Service**
Use EC2 Serial Console / SSM Session Manager to check if OS or service is down.
- ⑥ **Check CloudWatch Logs & Metrics**
Look for errors, high CPU, OOM, or application crashes.

Common Causes

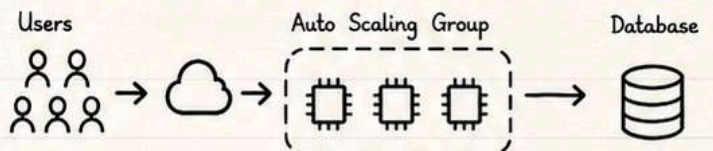
- Instance stopped or terminated.
- Security Group blocking the traffic.
- NACL denying the request.
- No public IP / wrong subnet route.
- OS crash, service down or port closed.
- Instance out of resources (CPU/RAM).
- Network ACL / Route table misconfiguration.



TIP:

Use Systems Manager Session Manager instead of SSH. It works even if the instance has no public IP!

- 15 Your application suddenly receives 10x more traffic than usual. How would AWS handle it?



How AWS Handles It

- ① **Route 53 (DNS)**
Handles high DNS queries and routes users to the closest healthy endpoint.
- ② **Elastic Load Balancer (ALB/ELB)**
Distributes traffic across multiple healthy instances automatically.
- ③ **Auto Scaling Group (ASG)**
Automatically launches new instances when traffic increases and scales in when traffic drops.
- ④ **CloudWatch + Alarms**
Monitors metrics (CPU, requests, latency) and triggers scaling policies.
- ⑤ **RDS / DynamoDB / ElastiCache**
Managed services scale storage, connections and throughput as needed.

Key Points

- ✓ AWS scales automatically; no manual action needed.
- ✓ ALB spreads load across multiple AZs.
- ✓ ASG adds capacity based on defined policies (CPU, requests, custom metrics).
- ✓ S3, CloudFront and AWS Global Infrastructure handle static content and reduce pressure.
- ✓ Application remains available and highly available.

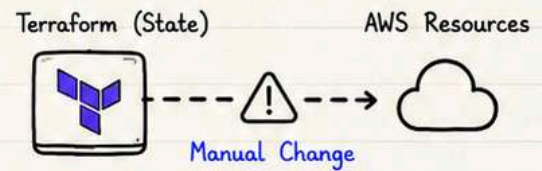


TIP:

- Design for scale from day one.
- Use Auto Scaling + Load Balancer.
- Keep application stateless.
- Monitor and set correct scaling thresholds.

TERRAFORM

- 16 Someone manually modified AWS resources managed by Terraform.
What problems can this create?



Problems Created

- ① **State Drift**
Terraform state no longer matches the real infrastructure.
- ② **Unexpected Updates**
Next plan may show changes you didn't expect.
- ③ **Risk of Resource Recreation**
Terraform may try to recreate or replace resources, causing downtime.
- ④ **Configuration Conflicts**
Manual settings may be overwritten by Terraform.
- ⑤ **Compliance & Audit Issues**
Deviations break governance and make audits fail.
- ⑥ **Team Confusion**
Engineers lose trust in the state and infrastructure visibility.

Best Practices to Avoid

- Never make manual changes to resources managed by Terraform.
- Use Terraform for all infrastructure changes.
- Enable version control for .tf files and state.
- Use Remote State (S3 + DynamoDB lock) for team consistency.
- Use workspaces for different environments.



TIP:

Always treat Terraform as the single source of truth for your infrastructure.
If it's not in code, it's not real!

- 17 Terraform detects infrastructure drift.
How would you resolve it?



How to Resolve Drift?

- ① **Detect Drift**
Run: `terraform plan` or `terraform plan -refresh-only`
- ② **Understand Difference**
Review the plan output and identify what changed manually.
- ③ **Decide the Correct Source of Truth**
Choose: Terraform code or actual infrastructure.
- ④ **Update Terraform Code (Preferred)**
If manual change is valid, update .tf files to match the real setup.
- ⑤ **Apply the Changes**
Run: `terraform plan` then `terraform apply` to sync state and infrastructure.
- ⑥ **Refresh State (If Needed)**
Run: `terraform refresh` to update state without changing infra.

Key Points

- Drift happens when real infra $\neq$ Terraform state.
- Plan -refresh-only helps detect drift safely.
- Always update code, not the other way around.
- Apply only after reviewing the plan.
- Use CI/CD to enforce consistent deployments.



TIP:

Automate drift detection using:

- `terraform plan -refresh-only` in CI pipelines
- AWS Config for continuous monitoring
- Alert on drift to catch issues early

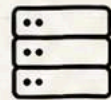
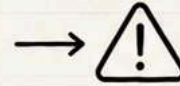
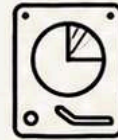
LINUX

18

Your production server reaches 100% disk usage. What would you check?

What I Would Check?

- ① Check Disk Usage Overview
df -h to see which filesystem is full.
- ↓
- ② Find Large Directories
du -sh /* | sort -hr
to find large folders.
- ↓
- ③ Check Large Files
find / -type f -size +100M -exec ls -lh {} \;
or use ncd for interactive view.
- ↓
- ④ Check Logs
Logs grow fast. Check /var/log
for large log files.
- ↓
- ⑤ Check Inodes
df -i to check inode usage.
If 100%, file creation fails.

Disk Usage
100%

Production Server

Common Causes

- Large log files not rotated
- Application dumping large files
- Temporary files not cleaned
- Docker images / containers taking space
- Database files / backups
- User uploads (images, videos, etc.)

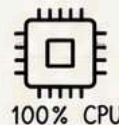


TIP:

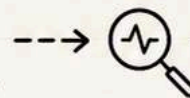
- Setup logrotate for all applications.
- Clean old files & backups regularly.
- Monitor disk usage with alerts.

19

A process is consuming 100% CPU.
How would you identify the root cause?



100% CPU

Root Cause
Identified

Steps I Would Follow

- ① Top/htop
top or htop to see which process is using CPU.
- ↓
- ② Check Process Details
ps -p <PID> -o pid,ppid,cmd,%cpu,%mem
Find the exact command.
- ↓
- ③ Check Parent Processes
ps -ef --forest | grep <PID>
See the parent-child chain.
- ↓
- ④ Check Logs
Application logs may show errors,
loops or heavy operations.
- ↓
- ⑤ Understand the Workload
Is it a bug, infinite loop, heavy query,
or scheduled job?

Common Reasons

- Application bug / infinite loop
- Heavy database query
- Cron job running too frequently
- Log file processing / parsing
- Resource intensive batch job
- Third party script / monitoring agent



TIP:

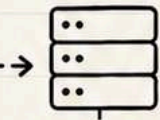
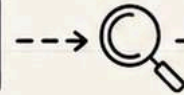
- Use nice and renice to reduce priority.
- Kill only if sure: kill -9 <PID>
- Fix the root cause, don't just kill it.
- Add monitoring & alerts for CPU spikes.

MONITORING

20

CPU utilization suddenly spikes across production servers.
How would you investigate?

CPU Spike



Production Servers

Investigation Steps



- ① Confirm the Spike
Check monitoring tools (CloudWatch, Prometheus, Grafana) to confirm the time, impact and affected servers.
- ② Identify the Process
Use top, htop or pidstat to find which process is consuming CPU.
- ③ Analyze the Process
Check command, threads, CPU %, and behavior. Is it expected or abnormal?
- ④ Check Logs & Errors
Look for errors, exceptions, retry loops or unusual activity.
- ⑤ Check Recent Changes
Deployments, config changes, new releases or batch jobs?
- ⑥ Mitigate & Monitor
Restart/scale if needed and monitor until usage returns to normal.

Common Causes

- Runaway process or infinite loop
- Memory leak causing CPU pressure
- High traffic or traffic surge
- Batch jobs / cron jobs running
- Inefficient database queries
- External dependency latency / retries
- Misconfiguration after deployment

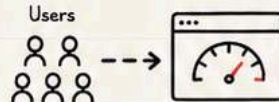


TIP:

- Always capture metrics for CPU, Memory, Disk and Network together.
- Correlate with logs and recent deployments before taking action.

21

Users report that the application is slow, but dashboards show everything is healthy. What would you check next?



Looks Healthy



Check Next

What I Would Check Next

- ① Check from User Perspective
Use real user monitoring (RUM) or synthetic checks from multiple regions.
- ② Check Latency at Each Layer
Analyze latency in DNS, CDN, Load Balancer, Application and Database.
- ③ Deep Dive into Application Traces
Use APM / distributed tracing (e.g. X-Ray, Datadog, New Relic) to find slow calls.
- ④ Check External Dependencies
Verify third party APIs, DB performance, payment gateways, etc.
- ⑤ Check Browser / Frontend Issues
Look for heavy JS, large assets, rendering blocks, or frontend errors.
- ⑥ Correlate Logs & Metrics
Check slow logs, error logs and custom metrics that may not be on dashboard.

Possible Reasons

- Slowness in a specific region or ISP
- High latency in external dependency
- Database slow queries
- Cache miss or cache not working
- Frontend performance issues (JS, CSS, Images)
- Network latency or packet loss
- Small % of users impacted (not visible in avg metrics)



TIP:

- Dashboards show averages, users feel the extremes.
- Always validate from user perspective.
- Use Tracing + Logs to find the real bottleneck.

PRODUCTION DEPLOYMENTS

22 Production goes down immediately after deployment. What are your next steps?



Next Steps

- ① **Confirm the Issue**
Check alerts, monitoring and user reports to confirm downtime.
- ② **Check Deployment Details**
Verify what was deployed, version, config changes and release notes.
- ③ **Rollback Immediately**
Rollback to the last known good version / previous deployment.
- ④ **Validate System Health**
Check services, DB, dependencies, logs and infrastructure.
- ⑤ **Communicate & Monitor**
Update stakeholders and monitor closely until system is stable.

Common Causes

- Bug in new release
- Configuration / environment changes
- Database / schema issues
- Missing dependencies
- Resource exhaustion
- Third party service failures



TIP:

- Always have a quick rollback plan ready.
- Use blue-green or canary deployments to reduce risk.
- Monitor key metrics & logs after every deployment.

23 A database migration fails during deployment. How do you avoid data loss?



How to Avoid Data Loss

- ① **Backup Before Migration**
Always take a full backup before running any migration.
- ② **Test in Staging**
Test migration scripts in a staging environment with real-like data.
- ③ **Use Transactions**
Wrap migration in transactions (if possible) to rollback on failure.
- ④ **Idempotent Scripts**
Write migration scripts to be idempotent (safe to run multiple times).
- ⑤ **Validate & Monitor**
Validate data after migration and monitor for anomalies.

Best Practices

- Use version-controlled migration tools (Flyway, Liquibase).
- Run migrations during low traffic windows.
- Keep rollback scripts ready.
- Use feature flags to decouple DB changes from app release.
- Document every migration and its impact.



TIP:

- Never run untested or manual migration in production.
- Always have a rollback plan + backup.
- Small, incremental migrations are safer than big-bang changes.

SECURITY

24

AWS access keys were accidentally committed to GitHub.

What actions would you take immediately?

Access Keys
Exposed

GitHub

Immediate Action



Immediate Actions

- ① **Invalidate the Keys**
Go to IAM → Security Credentials and deactivate/delete the keys.
- ② **Check for Misuse**
Review AWS CloudTrail logs for any API calls made using those keys.
- ③ **Rotate & Create New Keys**
Create new access keys for the user and update applications.
- ④ **Notify the Team**
Inform the security team and stakeholders about the exposure.
- ⑤ **Audit & Review**
Review IAM policies, permissions and repository history to prevent future leaks.

Why It's Critical

- Exposed keys can be used by anyone to access your AWS account.
- Attackers can create resources, steal data or increase costs.
- Quick action limits the blast radius.



TIP:

- Enable IAM Access Analyzer.
- Use least privilege permissions.
- Never use long-term keys in code. Prefer IAM Roles.

25

How would you secure an end-to-end CI/CD pipeline?

Security Best Practices

- ① **Secure Source Code**
Use private repositories, enforce branch protection and code reviews.
- ② **Manage Secrets Safely**
Store secrets in AWS Secrets Manager / Parameter Store. Never hardcode secrets.
- ③ **Least Privilege Access**
Use IAM roles with least privilege for CI/CD tools and services.
- ④ **Secure Build & Dependencies**
Scan dependencies and container images for vulnerabilities.
- ⑤ **Secure Deployment**
Use signed artifacts, approve deployments and enforce environment protections.
- ⑥ **Monitor & Audit**
Enable CloudTrail, CloudWatch and set up alerts for suspicious activities.

Code

Build

Test

Deploy

Monitor



Key Areas to Secure

- Code Repository
- Build Agents / Runners
- Secrets & Credentials
- Artifact Storage
- Deployment Targets
- Network & Access Controls
- Logging, Monitoring & Alerting



TIP:

- Automate security checks (SAST, DAST, IaC scan).
- Use OIDC + short-lived tokens instead of long-term credentials.
- Regularly rotate secrets and review permissions.
- Follow Shift-Left Security approach.

HIGH AVAILABILITY

- 26** One application server behind the load balancer is healthy but not receiving traffic. How would you troubleshoot?

How I Would Troubleshoot



- 1 Check LB Health Checks
Verify the target marked healthy and the health check path/port is correct.



- 2 Review Load Balancer Logs
Check access logs / target logs for any errors or 5xx.



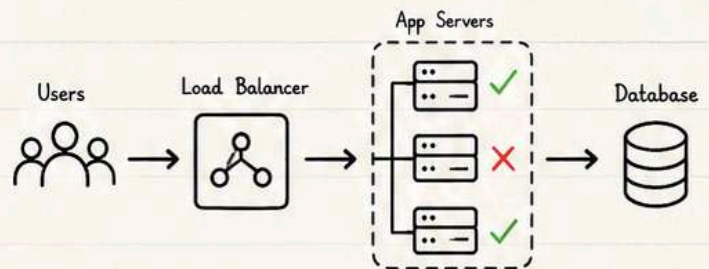
- 3 Check Target Group Settings
Ensure the target is registered and in "healthy" state.



- 4 Verify Routing Rules
Check sticky sessions, path-based routing or header rules affecting distribution.



- 5 Inspect Application & Network
Check app logs, firewall, security groups, NACLs and any rate limiting.



Possible Reasons

- Sticky sessions pinning users to other servers.
- Connection draining / slow start not completed.
- Path/host based routing excluding the server.
- Weight misconfiguration (weight = 0).
- Firewall / security group blocking traffic.
- Server resource limits causing slow responses.



TIP:

- Use CloudWatch metrics: HealthyHostCount, RequestCountPerTarget, HTTPCode_Target_5XX.
- Test directly on the server using its private IP.
- After changes, wait for new connections to distribute.

- 27** Your Kubernetes control plane fails unexpectedly. What happens next?

What Happens?



- Worker nodes keep running the existing pods.



- Running applications continue to serve traffic.



- No new deployments or scaling can happen.



- Cannot create/update resources (pods, svc, cm, etc.).



- Logs/metrics may still be available from nodes.

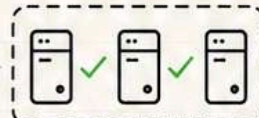


- If etcd is also down and not replicated, cluster state may be lost.

Control Plane (Down)



Workers Continue



Applications Keep Running



Recovery / Next Steps

- 1 Check control plane component: API Server, etcd, Scheduler, Controller Manager.
- 2 Restore from backup if etcd data is lost.
- 3 Redeploy control plane (HA setup should auto-recover).
- 4 Verify cluster connectivity: `kubectl get nodes`
- 5 Confirm workloads, controllers and networking are healthy.



TIP:

- Always run control plane in HA across multiple AZs.
- Enable etcd backups and test restore regularly.
- Monitor control plane components with alerts.
- Keep runbooks ready for disaster recovery.

DEPLOYMENT STRATEGIES

28

When would you choose Blue-Green Deployment instead of Rolling Deployment?

When to Use Blue-Green?

- Need zero-downtime with instant rollback.
- Major changes / risky releases.
- Database is backward compatible or decoupled.
- Easy traffic switch (via Load Balancer / DNS / Router).
- Environment can be duplicated (double the infra cost is ok).



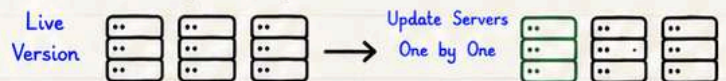
TIP:

- Blue-Green gives fast rollback: just switch traffic back.
- Rolling is good for small changes and lower infra cost.

Blue-Green Deployment



Rolling Deployment



Why Not Always Use Rolling?

- Harder to rollback (need to roll back each server).
- During rollout, old and new versions run together – risk of compatibility issues.
- User experience may be impacted during rollout.
- Not ideal for large / breaking changes.

Advantages of Blue-Green

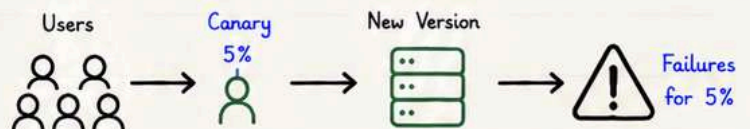
- Zero downtime.
- Fast rollback.
- Easy to test the new version before switching.
- Stable and safe for critical systems.

29

A Canary deployment causes failures for only 5% of users. How would you investigate?

Investigation Steps

1. Check Monitoring & Alerts
Review dashboards, error rates, latency, CPU, memory for canary instances.
2. Analyze Logs
Check app logs, access logs, and error logs for canary pods/instances.
3. Compare Metrics
Compare canary vs stable version metrics. Look for anomalies.
4. Check Dependencies
Verify DB, cache, external APIs, message queues, and 3rd party services.
5. User Segmentation
Identify patterns: device, region, browser, ISP, user group?
6. Rollback or Limit Traffic
If issue confirmed, rollback canary or reduce canary traffic to 0%.



Possible Root Causes

- New code bug or null pointer exception
- Performance degradation (slow queries, memory leak)
- Incompatible change with data / schema
- Feature flag misconfiguration
- External service timeout / rate limits
- Environment / config difference



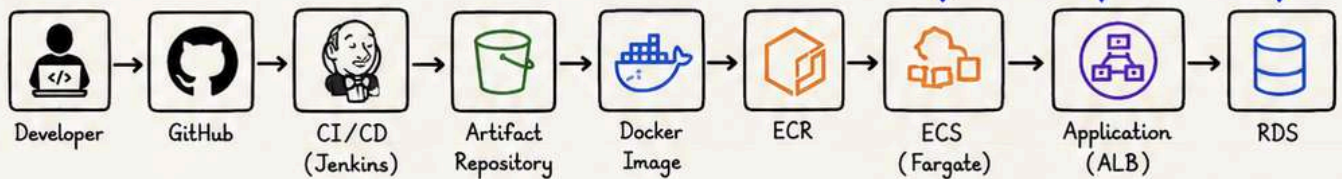
TIP:

- Use distributed tracing to follow full request path.
- Check release notes and recent change history.
- Communicate with the team and update status.
- Never ignore small % failures in canary – they can become big after full rollout.

SYSTEM DESIGN

- 30** Design a production-ready DevOps pipeline for a Java Spring Boot microservice running on AWS.

High Level Architecture



Pipeline Stages

- 1 **Code Commit**
• Developer pushes code to GitHub.
- 2 **Build**
• Jenkins pulls code and builds the project using Maven.
- 3 **Unit Tests**
• Run JUnit tests and generate reports.
- 4 **Code Quality Check**
• Analyze code using SonarQube.
- 5 **Security Scan**
• Scan dependencies & image using Trivy / OWASP.
- 6 **Package & Artifact**
• Create JAR/WAR and store in S3 (Artifact Repository).
- 7 **Build & Push Docker Image**
• Build Docker image and push to Amazon ECR.
- 8 **Deploy to ECS (Fargate)**
• Deploy new task definition and update service.
- 9 **Health Check**
ALB health checks ensure only healthy tasks receive traffic.
- 10 **Rollback (If Needed)**
Auto rollback using previous task definition.

AWS Services Used

- Code: GitHub
- CI/CD: Jenkins
- Artifact Storage: Amazon S3
- Container Registry: Amazon ECR
- Orchestration: Amazon ECS (Fargate)
- Load Balancer: Application Load Balancer (ALB)
- Database: Amazon RDS (Multi-AZ)
- Monitoring: Amazon CloudWatch
- Tracing: AWS X-Ray
- Logging: CloudWatch Logs
- Audit: AWS CloudTrail
- Secrets: AWS Secrets Manager



TIP:

- Use separate environments: Dev → Staging → Prod
- Store secrets in AWS Secrets Manager.
- Enable Auto Scaling on ECS Service.
- Use Blue/Green or Canary for safe deployments.

Key Features

- Automated build, test, scan and deployment
- Immutable infrastructure using containers
- High availability with ALB + ECS + RDS Multi-AZ
- Centralized logging and monitoring
- Secure and auditable pipeline
- Easy rollback and zero/low downtime deployments

Best Practices

- ✓ Keep images small
- ✓ Scan early & often
- ✓ Infrastructure as Code (Terraform/CloudFormation)
- ✓ Enforce least privilege
- ✓ Monitor everything
- ✓ Document and version pipeline code

Flow Summary

Code → Build → Test → Scan → Package → Docker Image → Push to ECR → Deploy to ECS → ALB → RDS
+ Monitor, Log, Trace, Secure

RAPID FIRE

1 Docker Image vs Container

Docker Image



- Read-only template with application and dependencies.
- Built using Dockerfile.
- Stored in a registry (e.g., Docker Hub).
- Immutable - same image creates same container.

VS

Docker Container



- Runnable instance of an image.
- Has its own writable layer.
- Can be started, stopped, deleted.
- Isolated process with its own network & filesystem.

2 Deployment vs StatefulSet

Deployment



- For stateless applications.
- Pods are interchangeable.
- No guaranteed order of deployment.
- Use ReplicaSet under the hood.

VS

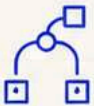
StatefulSet



- For stateful applications.
- Each pod has a stable identity.
- Ordered deployment & scaling.
- Stable storage with Persistent Volumes.

3 ALB vs NLB

ALB (Application Load Balancer)



- Operates at Layer 7 (HTTP/HTTPS).
- Routes traffic based on content (host, path, headers).
- Supports WebSockets, redirects, authentication.
- Ideal for web applications.

VS

NLB (Network Load Balancer)



- Operates at Layer 4 (TCP/UDP).
- Routes traffic based on IP & port.
- Ultra high performance, static IP.
- Ideal for high-throughput, low-latency applications.

4 Horizontal vs Vertical Scaling

Horizontal Scaling (Scale Out)



- Add more instances/pods.
- Distributes load across instances.
- Improves availability & fault tolerance.
- Example: Auto Scaling, Replica increase.

VS

Vertical Scaling (Scale Up)



- Increase resources (CPU/RAM) of existing instance.
- Simple to implement.
- Has hardware limits.
- Example: Upgrade instance type.

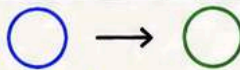
5 Rolling vs Blue-Green vs Canary

Rolling Deployment



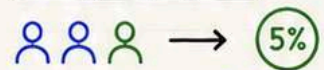
- Replace old version gradually.
- Downtime: Minimal.
- Risk: Medium.
- Simple & standard approach.

Blue-Green Deployment



- Two identical environments.
- Switch traffic from Blue to Green.
- Downtime: None (very low).
- Risk: Low, but infra cost doubles.

Canary Deployment



- New version to a small % of users.
- Monitor & validate.
- Downtime: None.
- Risk: Low, better early feedback.

INTERVIEW TIPS

① Explain your troubleshooting process.



- Follow a structured approach.
- Identify the issue, collect information, analyze, and isolate the root cause.
- Explain the steps you take and why.
- Mention tools and commands you use.

② Mention logs, metrics, and monitoring.



- Check application logs for errors.
- Monitor metrics: CPU, Memory, Disk, Latency, Error Rates, Throughput.
- Use tools like CloudWatch, Prometheus, Grafana, ELK to gain insights.

③ Discuss rollback and recovery.



- Have a clear rollback strategy.
- Use versioning, backups and snapshots.
- Minimize downtime and data loss.
- Test rollback process regularly.

④ Highlight scalability and security.



- Design for scale: Auto Scaling, Load Balancing, Caching, Stateless services.
- Follow security best practices: IAM least privilege, Encryption, Secrets Management, Network Security.

⑤ Justify your technical decisions.



- Explain why you chose a particular tool, architecture or approach.
- Compare alternatives and trade-offs.
- Show that you think about performance, cost, reliability and maintainability.

COMMON MISTAKES

① Memorizing commands without understanding concepts.



- Leads to confusion when something changes.
- Focus on understanding how and why things work.

② Ignoring production impact.



- Small changes can cause major outages.
- Always evaluate risk before making changes.

③ Forgetting rollback strategies.



- No rollback plan = longer downtime.
- Always have a tested rollback plan before deployment.

④ Overlooking monitoring and logging.



- You can't fix what you can't see.
- Monitoring and logs are critical for quick issue detection and resolution.

⑤ Providing tool-specific answers without explaining the reasoning.



- Interviews test your problem-solving skills, not just tool usage.
- Explain the why, approach and trade-offs behind your answers.